
Implicitly Learning Physics from Videos using a Joint-Embedding Predictive Architecture

Raif T. Olson

Thayer School of Engineering
Dartmouth College
Hanover, NH 03755

raif.t.olson.th@dartmouth.edu

Andrei Gerashchenko

Thayer School of Engineering
Dartmouth College
Hanover, NH 03755

andrei.gerashchenko.th@dartmouth.edu

1 Abstract

Existing self-supervised vision models are not good at learning physics implicitly, which some researchers believe is due to them learning to directly reproduce pixels rather than learning to reproduce semantic information. The Joint Embedding Predictive Architecture was introduced by Assran et al. (2023) to train image encoders to encode semantic information that could be used to reproduce the semantic information that was masked out of an image. The V-JEPA architecture by Bardes, A. et al. (2024) expanded this to video to teach vision models to learn information useful for modeling physics. More recently Drozdov, K. et al. (2024) modified the JEPA architecture to generate frames in simple datasets like Moving MNIST, and then train a linear probe to extract physical metrics like velocity to evaluate the model’s ability to understand physics just from self-supervised training.

https://github.com/rolson24/OpenSTL_with_JEPA

2 Introduction

Recent work in papers like Video-MAE has found that pre-training vision encoders like ViT[3] and ViViT[1] using masking strategies lead to high-quality representations that can then be fine-tuned to achieve state-of-the-art on downstream tasks. [8] Unfortunately, these methods require hundreds of thousands of images and videos for training, as well as hundreds of millions of training steps for these models to fully converge[2], making these methods out of reach for researchers without huge computational budgets.

The JEPA architecture proposed by Bardes et al. attempts to show that by performing masked pre-training to predict the masked representations from the unmasked representations, a foundation vision encoder can be trained with significantly fewer training steps than other self-supervised training methods. Unfortunately, due to the architecture of JEPA, it is possible that the encoder learns to simply output a trivial solution in a circumstance called "representation collapse." In Drozdov, K. et al.’s work, they propose using a Variance-Covariance regularization term to prevent this collapse. They pre-train a small JEPA architecture on a simple video dataset and then use linear probing on the predicted representations to extract the velocity of the moving digit to demonstrate the model’s ability to learn physics from self-supervised training on videos.

In our work, we reproduce the SimVP and Swin transformer JEPA model from Drozdov et al. and introduce a new dataset that simulates more advanced physics than Moving-MNIST called Bouncing Shapes. This dataset is created by simulating one of three possible shapes (circle, square, and triangle) in a 2-dimensional environment with gravity, air resistance, surface friction, and slightly inelastic collisions. The dataset consists of 10,000 simulations consisting of 20 frames, with each showing a random shape with a random position, rotation, and initial velocity. In order to ensure that the 20-frame sequences accurately depict the physics model, simulation speed is set to show one or two bounces over the full sequence.

3 Related Work

3.1 Joint Embedding Predictive Architectures

Existing work has shown that aligning the representation space of a Vision Mamba encoder to a frozen CLIP-ViT encoder on unmasked image tokens can be effective. However, both Li et al. [5] and Liu et al. [6] found that this architecture did not scale well to larger model sizes due to their use of smaller video datasets, leading to overfitting issues.

In an attempt to address these challenges, we explored the V-JEPA (Video-based Joint-Embedding Predictive Architecture) training method recently introduced by Bardes et al. [2]. V-JEPA builds upon the principles of masked modeling from VideoMAE, which already employs masked video modeling for video analysis, but specifically enhances it for improved training efficiency. Unlike VideoMAE, which focuses on reconstructing the original video frames from masked inputs, V-JEPA predicts the masked regions in the representation space. This approach reduces the computational burden and accelerates convergence, making the training process more efficient and effective.

One significant benefit of V-JEPA is its potential to reduce the likelihood of overfitting by shortening the training time required on larger datasets. Traditional masked autoencoder training sessions for vision encoders, such as those used in VideoMAE, typically require extensive computational resources and prolonged training periods [8][2]. These extended durations make training larger Video Mamba models out of reach for computationally limited researchers.

V-JEPA addresses these issues with a unique training mechanism. By using a context encoder to process partially visible video frames and a target encoder to provide 'ground truth' representations, the predictor learns to infer missing information efficiently. This setup not only stabilizes the learning process but also accelerates convergence. The target encoder, being a moving weighted average of the context encoder, ensures stable and consistent learning.

3.2 Swin

The Swin transformer was introduced by Liu et al (2021) [7] as a way to process images and videos by performing "shifted window attention" across patches of the image or video. This method allows each patch to "see" the entire image while reducing memory usage. Each Swin block computes attention matrices on fixed windows, and subsequent blocks use shifted windows so that patches attend to different sets of neighbors. We use the Swin transformer to attend across the representations of the input frames to predict the representations of the future frames.

3.3 SimVP

As a demonstration of the capabilities of encoder-decoder architecture for predicting future frames, Gao et al. (2022) introduced the Simple Video Prediction model which consists of an encoder, a translator, and a decoder. Their encoder is a spatial 2D convolutional model with layer norms and leaky ReLU activation functions that processes each frame individually, and the decoder is a 2D convolution model that upsamples the features and performs a 2D convolution with a group norm and then a leaky ReLU. We use the encoder and decoder from this paper in our architecture.

4 Proposed Approach

We directly use the architecture described by Drozdov, K. et al. along with the VJ-VCR loss function to prevent representation collapse in our model.

Our revised model is built on the following key components:

4.1 Full-Frame Input Strategy

Rather than masking out portions of the input, our model is provided with two contiguous segments of video:

- **Context Segment:** The first 10 frames are fed directly (unmasked) into the context encoder.

- **Target Segment:** The subsequent 10 frames are processed by the target encoder, whose output serves as the “ground truth” representation.

The predictor is then trained to map the latent representations from the context encoder to those of the target encoder.

4.2 Encoder Architecture and Spatial Feature Extraction

In previous JEPA architectures, the representation collapse problem was mitigated by formulating the target encoder as an exponential moving average of the context encoder. In this model, we instead make the context and the target encoder the exact same and prevent representation collapse with the VCR loss regularization.

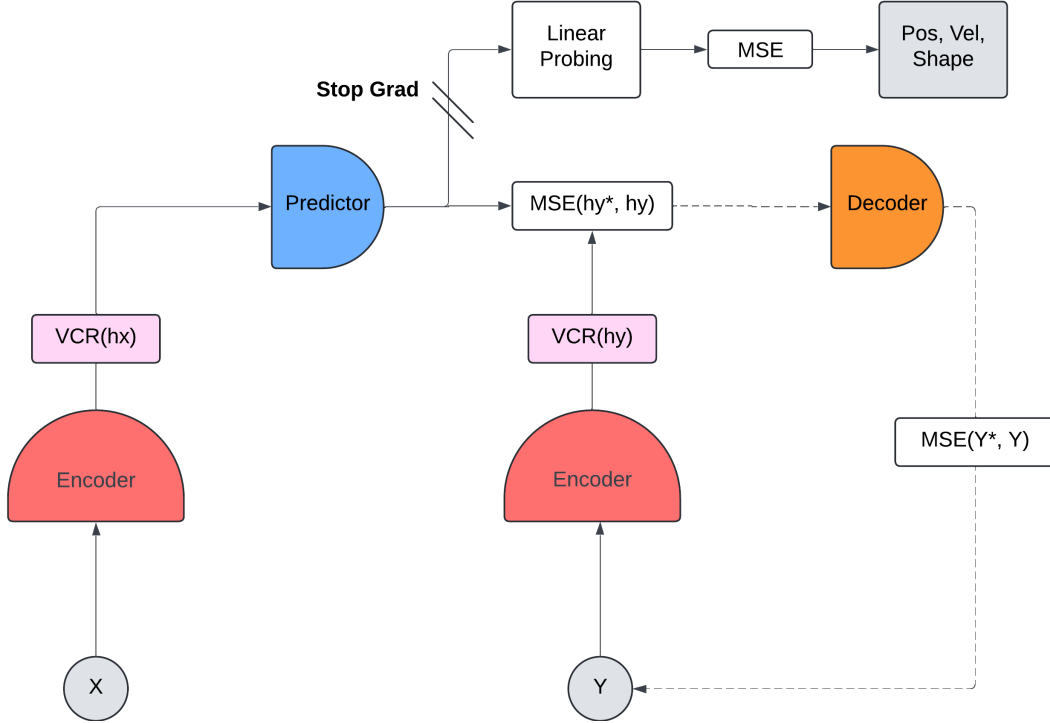


Figure 1: The figure illustrates the architecture of the V-JEPA (Video-based Joint-Embedding Predictive Architecture) model. The process begins with input video frames X as 64 x 64 single channel images. These frames are passed through the encoder independent of each other to get their embedding space representation. During pre-training, the context encoder processes the future frames, to get their embedded representations. The predictor attempts to predict (reproduce) the embeddings of the future frames from the embeddings of the input frames. An optional decoder is trained to decode the representations back to images. The main loss is a Mean Squared Error loss computed between the predicted future representations and the "true" future representations. The VCR regularization terms are computed on both the input representations and the future representations to prevent representation collapse. If a decoder is being trained, an additional loss is computed between the decoded frames and the target frames. When evaluating the model for learned physics, a linear probing layer is added on top of the predictor to learn object position, velocity, and shape.

4.3 Predictor and Variance-Covariance Regularization

The predictor module takes as input the latent representations generated by the context encoder and aims to recreate the representations of the target encoder. To prevent representation collapse—a common issue in joint-embedding architectures—we integrate a variance-covariance regularization (VCR) term into the training objective. This regularization encourages each feature dimension to exhibit sufficient variance while minimizing redundancy across different dimensions.

The VCR loss regularization is given by:

$$\mathcal{L}_{\text{VCR}} = \alpha l_{\text{var}}[H] + \beta l_{\text{cov}}[H],$$

where $l_{\text{var}} = \frac{1}{Td} \sum_{t=1}^T \sum_{k=1}^d \max(0, \tau - \sqrt{\text{Var}(H_{t,k} + \epsilon)})$ is the variance loss (designed to force each feature to have a variance greater than ϵ) and $l_{\text{cov}} = \frac{1}{Td} \sum_{t=1}^T \sum_{i \neq j} [\text{Cov}(H_{t,:})]_{i,j}^2$ is the covariance regularization term which forces the covariance between features to have low covariance so that each feature encodes a different piece of information. α is the weighting of the variance term and β is the weighting of the covariance term.

4.4 Training Mechanism

During training, a continuous video clip is split into two segments: the context frames and the target frames. The context encoder processes the first 10 frames to produce latent embeddings that capture the underlying physics and dynamics. Simultaneously, the target encoder processes the subsequent 10 frames to generate stable representations. The predictor then learns to map the context embeddings to the target embeddings by minimizing the combined loss function.

This full-frame input strategy eliminates the need for complex masking mechanisms, reducing both computational overhead and potential information loss. At the same time, by leveraging complete frame information, the model is better positioned to learn rich, high-level representations of the physical interactions within the video.

The model is then trained by optimizing the full loss function:

$$\mathcal{L} = \text{MSE}(h_y^*, h_y) + \mathcal{L}_{\text{VCR}}(H_x) + \mathcal{L}_{\text{VCR}}(H_y) + \text{MSE}(Y^*, Y)$$

Where h_y^* are the predicted representations, h_y are the "true" future representations, Y^* are the decoded frames, and Y are the true future frames.

5 Experimental Results

5.1 Implementation Details

To conduct our experiments, we use the OpenSTL code and modify it to implement the JEPA architecture to train our models. For hyperparameters, we use the recommended pre-training parameters from Drozdov, K. et al. [4] with an α of 0.1 and a β of 0.1. Our experimental code can be seen in this GitHub repository https://github.com/rolson24/OpenSTL_with_JEPA.

We create our bouncing shapes dataset by using a simple Python physics simulation of a shape bouncing around a 256 x 256 pixel frame. The shape has gravity applied to it and has imperfectly elastic collisions with the edges of the frame to simulate more accurate physics. The simulation also models how the shape would rotate based on its angle of collision with the edges of the frame. We randomly sample the shape's initial (x, y, θ) position, and (dx, dy, d θ) velocity and have an even distribution of shapes (triangle, square, circle). We simulate each run for 20 frames and save the frames as single channel 64 x 64 images in a compressed NumPy array for training. This dataset is designed to evaluate the model's ability to learn physics from videos without having other distractions or artifacts that would be present in real datasets. Tools for generating this dataset are available in the GitHub repository mentioned above, which can be used to create longer sequences with different parameters, and can be extended to simulate more complex objects or environments.

5.2 Intermediate Results

To date, we have conducted experiments by pre-training the model on both our bouncing shapes dataset and on the Moving MNIST dataset. Due to limited computational resources, we were unable to conduct linear probing on the pre-trained encoder-predictor to extract the physical quantities like shape, position and velocity.

We were able to evaluate the model by predicting the next 10 frames in a held off test set. We can see the results in Figure 3 where the model is able to predict the next 10 frames quite accurately, aside from in rows 2 and 3 where the model has a different interpretation of the bouncing behavior.

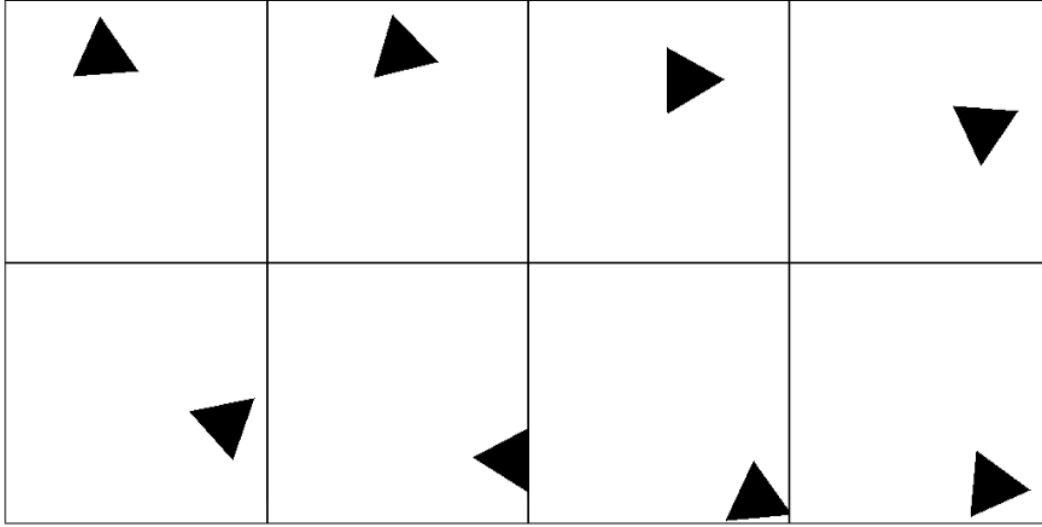


Figure 2: An example dataset video sequence. The frames progress from left to right and then onto the second row.

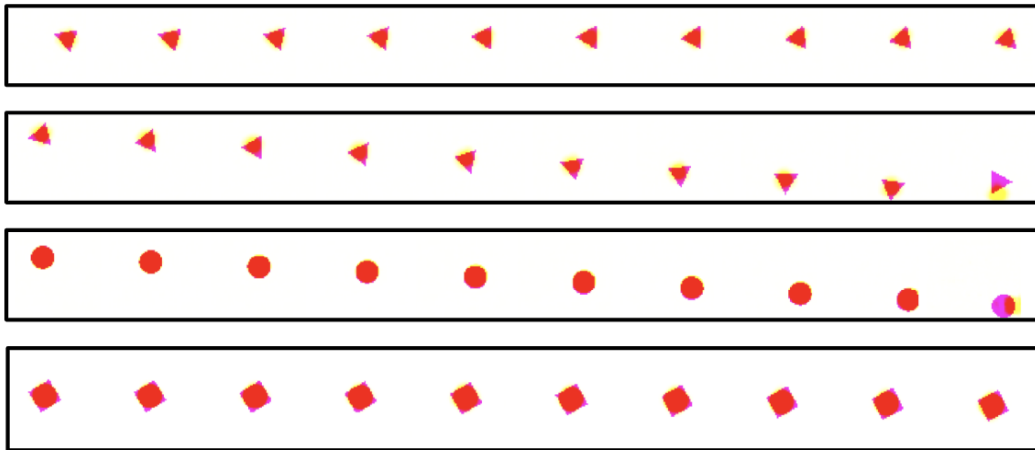


Figure 3: These are some examples of the model predicting the next 10 frames. Each row is a different run. The predicted frames are overlaid with the true frames. The predicted pixels are in yellow, the true pixels are in purple, and the red is the overlap. As we can see in row 1 and 4, the model does quite a good job of tracking the rotation of the shapes, as well as the velocity. In rows 2 and 3 we can see that the model is still confused by shapes bouncing, which we believe is because within 20 frames the model only gets to see at most 2 bounces during training. Longer training sequences may resolve this problem by providing more examples of repeated bounces.

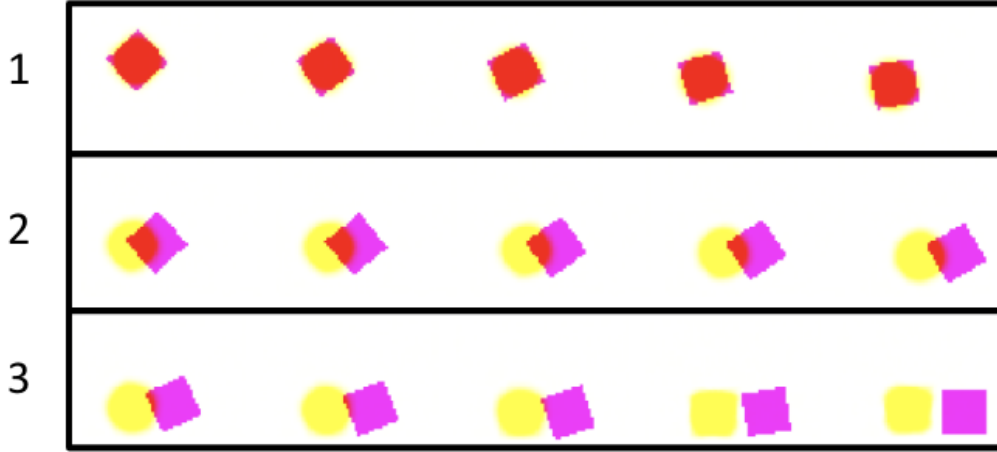


Figure 4: This shows an experiment where we simulated the environment for 100 frames and fed the predicted frames output from the model back into the model to predict the next 10 frames in a loop to predict 90 of the 100 simulation frames. The top row are the first 5 predicted frames, the 2nd row are frames 45-50, and the 3rd row are frames 85-90.

We also tested the model on predicting 90 frames only from the first 10 frames. This was done by feeding the previously predicted frames back into the model as input to predict the next 10 frames. We can see in Figure 4 that the model starts off with an accurate prediction, (correct shape, correct position), however as the simulation goes on the model lags behind the true frames the object begins to turn into a circle, changing back to a square at the end of the simulation after the object settles. We believe this change in shape is due to the model being aware of the correct shape the entire time, but displaying it as a circle because of the shape’s rotation. This is shown by the object’s correct shape being restored once the object is resting on the ground and has no rotation.

6 In Progress

To further evaluate the model’s ability to learn the physics of the simulation, we trained a linear probe head on the predicted representations of the future frames to extract the 7 physical attributes of the shape in each frame. These attributes consisted of x , y position, θ rotation, x and y velocities, $d\theta$ angular velocity, and the class of the shape (triangle, square, or circle). For each attribute we trained a separate linear layer with an input size of 1024 (number of features per embedded frame) and an output size of 1, with the exception of the class head which has 3 outputs for the 3 possible classes. We used an MSE criterion as our loss function for the scalar attributes and a cross entropy loss for the classification head. The results of this linear probing (MSE of 20.97) indicate that the representations stored in the individual frames do not do a good job of encoding the physical attributes like we expected. Instead we hypothesize that the predictor’s attention mechanism is directly encoding the physical quantities for velocity and position, and the representations are merely scaling those quantities.

7 Conclusion

In this work, we introduced a novel approach to implicitly learning physics from videos by integrating a Joint-Embedding Predictive Architecture with a full-frame input strategy and robust variance-covariance regularization. Our method builds on the strengths of both JEPA and VideoMamba, allowing the model to capture intricate physical dynamics—such as velocity, rotation, and collision behavior—while reducing training iterations and mitigating overfitting.

Experiments on both the Moving MNIST dataset and our newly introduced Bouncing Shapes dataset demonstrated the model’s ability to accurately predict future frames and implicitly learn meaningful physical representations. Although the model successfully tracked dynamics in many

cases, challenges remain in achieving long-term prediction stability and consistent shape decoding, particularly in scenarios with rapid or complex interactions.

Future work will focus on refining the predictor architecture, optimizing hyperparameters, and extending evaluations to more diverse and realistic video datasets. We believe that our findings lay a strong foundation for advancing efficient video representation learning and pave the way for more robust physics-based video analysis systems.

References

- [1] Anurag Arnab, Mostafa Dehghani, Georg Heigold, Chen Sun, Mario Lučić, and Cordelia Schmid. Vivit: A video vision transformer, 2021.
- [2] Adrien Bardes, Quentin Garrido, Jean Ponce, Xinlei Chen, Michael Rabbat, Yann LeCun, Mido Assran, and Nicolas Ballas. V-JEPA: Latent video prediction for visual representation learning, 2024.
- [3] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale, 2021.
- [4] Katrina Drozdov, Ravid Shwartz-Ziv, and Yann LeCun. Video representation learning with joint-embedding predictive architectures, 2024.
- [5] Kunchang Li, Xinhao Li, Yi Wang, Yinan He, Yali Wang, Limin Wang, and Yu Qiao. Video-mamba: State space model for efficient video understanding, 2024.
- [6] Yue Liu, Yunjie Tian, Yuzhong Zhao, Hongtian Yu, Lingxi Xie, Yaowei Wang, Qixiang Ye, and Yunfan Liu. Vmamba: Visual state space model, 2024.
- [7] Ze Liu, Yutong Lin, Yue Cao, Han Hu, Yixuan Wei, Zheng Zhang, Stephen Lin, and Baining Guo. Swin transformer: Hierarchical vision transformer using shifted windows, 2021.
- [8] Zhan Tong, Yibing Song, Jue Wang, and Limin Wang. Videomae: Masked autoencoders are data-efficient learners for self-supervised video pre-training, 2022.